EXPLAINABLE NEWS RECOMMENDATION

A news recommender trained on *real clicks*, that explains every pick.

TopicLens recommends the news articles each reader is most likely to want, learned from real Microsoft News click logs, and shows exactly why each article was recommended.

1. What is it for?

News apps decide what people read, but their models are black boxes: readers cannot see why a story was chosen or correct a wrong guess. TopicLens is a university research project, built on Liu et al. (2024), *Topic-Centric Explanations for News Recommendation* (ACM TORS 3(2)). It aims to recommend accurately **and** transparently, so recommendations can be trusted and steered.

01
Readers
A feed ranked for them, with the reasons one tap away and controls to push back.

02
Students & researchers
A complete, inspectable pipeline: real data, features, 14 models, standard metrics.

03
Reviewers
Every number is labelled as measured here, taken from the paper, or not yet evaluated.

2. What does it do?

Ranks a personal feed from four kinds of evidence:

- **Reading & clicking history**: opening a story counts as a click and joins the history.
- **Preferred sections** picked at sign-up and editable later.
- **Interaction with earlier recommendations**: stories passed over above a click move down.
- **Explicit feedback**: More / Less like this, Not interested, topic sliders and mutes.

Explains each pick on request: the score split into its signals (they add up exactly), the past reads behind it, and the words that matched.

Web app: React + TypeScript, runs in the browser; sign in with Google, email, as a guest or as a real anonymised demo reader. **ML**: Python (scikit-learn, LightGBM, PyTorch + CUDA).

3. How does it recommend?

Article text	**Content vectors**	**10 features**	**Trained classifier**	**Ranked feed**
Title + abstract of each article.	30 NMF topics and a 100-d LSA text embedding.	per reader × article: topic & text match, section affinity, popularity, freshness…	Logistic regression predicts P(click).	Sorted by P(click), adjusted by the reader's feedback.

The reader profile is the average of their last 50 reads, exactly as during training.

> **Does it use KNN or SVM?** Not in the app. We trained and tested **KNN, a linear SVM, a kernel (RBF) SVM, a random forest, LightGBM, logistic regression, a neural network (NRMS, on a GPU) and a hybrid** on the same data. The app ranks with **logistic regression** (AUC 63.84): it ties for best of the single models (LightGBM 63.83, linear SVM 63.74) and its score splits exactly into per-signal reasons. KNN scored lowest of the trained models (AUC 55.66). The most accurate model overall is the **hybrid** (AUC 66.60), reported but not run in the browser because it needs the neural network.

DATA

4. The dataset: *MIND-small*

MIND, the Microsoft News Dataset (Wu et al., ACL 2020), contains real, anonymised logs from Microsoft News: for each visit ("impression"), the articles a user was shown and which ones they clicked, plus that user's earlier click history. It is the benchmark used by the paper this project builds on. License: Microsoft Research License (non-commercial research). We verified our copy against Microsoft's published row counts (156,965 / 73,152 impressions; 51,282 / 42,416 articles).

65,238	94,057	230,117	8.58M
news articles (title, abstract, category, sub-category)	anonymised users	impressions with clicks	articles shown; 4.05% clicked

5. Train / test split

The split is by **time**, so the model is always tested on the future, as it would be in use. MIND's own test labels are not public, so the official dev set is our test set.

Training · 9–13 Nov 2019	Validation · 14 Nov	Test · 15 Nov 2019
126,695 impressions · 4.62M candidate rows	30,270 impr. 1.22M rows	73,152 impressions · 2.74M rows (never seen)

- **Training**: fits every model. **Validation** (last training day): early stopping and model selection only.
- **Test**: used once, to report every number on page 3.
- The hybrid is trained on the validation day, which the neural network never saw, so its neural-score input is an honest out-of-sample prediction.
- Heavy models were fitted on random samples of the training rows (KNN 60k, kernel SVM 150k, linear SVM and random forest 400k), then scored on all 2.74M test rows.

6. Features (no label leakage)

FEATURE	WHAT IT MEASURES
Topic match	Dot product of the user's mean NMF topic vector (last 50 reads) and the article's topics (30 topics from TF-IDF).
Text similarity	Cosine between the user's mean LSA embedding (100-d) and the article's.
Section / sub-section affinity	Share of the user's reads in the article's category and sub-category.
Earlier-day clicks & CTR	Clicks and smoothed click-through rate from **strictly earlier days** of the training log only.
Prior exposure, freshness	Times shown before this moment; hours since first appearance (from timestamps, never labels).
History length, impression size	How much the user has read; how many articles were shown together.

Which features matter most (LightGBM, share of split gain)

- 19% Click-through rate on earlier days (smoothed)
- 15% Article category
- 13% Share of your reads in this subcategory
- 11% Text similarity to your history (LSA embedding)
- 10% Number of articles shown together (log)
- 9% Hours since the article first appeared (log)

The neural model (NRMS-style) learns from text instead: a MiniLM sentence encoder for articles and self-attention over the user's last 50 reads, trained on 189,519 clicks with 4 sampled negatives each, 4 epochs on an RTX 3050 (8 min).

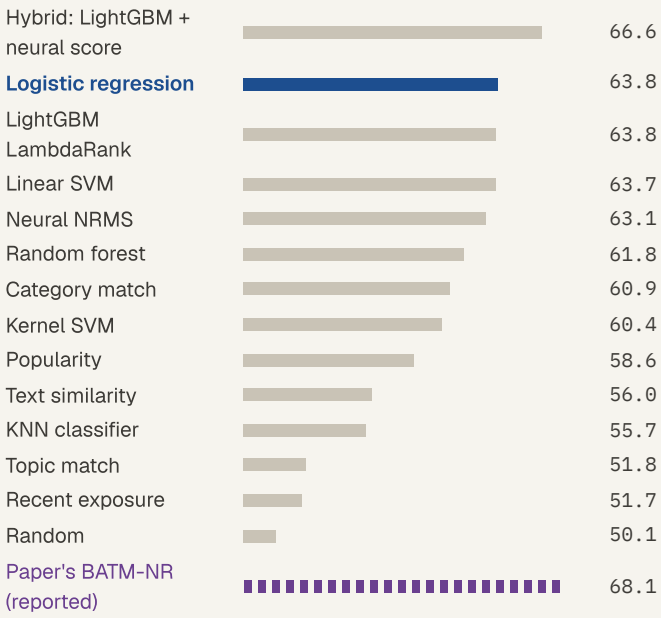
RESULTS

7. Performance on *73,152 test visits*

Metrics follow MIND's definitions, computed per visit and averaged; ± is the 95% confidence interval from 500 bootstrap resamples. Highlighted: the model the app uses.

MODEL	TYPE	AUC	MRR	NDCG@5	NDCG@10
Hybrid: LightGBM + neural score	Hybrid (stacked)	66.60 ±0.20	32.05 ±0.23	35.47 ±0.26	41.64 ±0.23
Logistic regression	Linear classifier	63.84 ±0.21	30.56 ±0.23	33.54 ±0.27	39.61 ±0.23
LightGBM LambdaRank	Gradient-boosted trees	63.83 ±0.22	30.58 ±0.21	33.26 ±0.24	39.46 ±0.23
Linear SVM	Support vector machine	63.74 ±0.23	30.56 ±0.22	33.52 ±0.27	39.59 ±0.23
Neural NRMS (GPU, MiniLM news encoder)	Neural network (GPU)	63.14 ±0.22	29.75 ±0.23	32.20 ±0.26	38.64 ±0.23
Random forest	Tree ensemble	61.79 ±0.21	29.26 ±0.21	31.95 ±0.25	38.15 ±0.23
Category match	Rule (baseline)	60.94 ±0.20	29.24 ±0.21	31.71 ±0.25	37.64 ±0.23
Kernel SVM (RBF, Nystroem)	Support vector machine	60.40 ±0.21	27.86 ±0.22	30.27 ±0.26	36.82 ±0.23
Popularity (earlier-day clicks)	Rule (baseline)	58.61 ±0.21	27.24 ±0.19	28.66 ±0.24	34.73 ±0.22
Text similarity (LSA)	Rule (baseline)	56.03 ±0.22	25.61 ±0.21	26.89 ±0.26	33.39 ±0.23
KNN classifier (k=50)	Nearest neighbours	55.66 ±0.24	25.97 ±0.20	27.85 ±0.25	34.00 ±0.22
Topic match (NMF)	Rule (baseline)	51.83 ±0.20	22.34 ±0.20	22.41 ±0.23	29.35 ±0.22
Recent exposure	Rule (baseline)	51.69 ±0.21	23.00 ±0.20	24.40 ±0.22	30.47 ±0.21
Random	Rule (baseline)	50.07 ±0.21	21.91 ±0.18	22.38 ±0.21	28.66 ±0.21
Paper's BATM-NR (from its Table 5)	Neural, reads full text	68.13	32.62	36.36	42.46

AUC by model (50 = random)



What the metrics mean

- **AUC**: chance that a clicked article ranks above an unclicked one.
- **MRR**: how high the first click sits (1/rank, averaged).
- **nDCG@k**: how much of the clicked content lands in the top k.

> **Why not plain "accuracy"?** Only 4.0% of shown articles are clicked, so predicting "no click" for everything is 96.0% accurate yet useless. Recommenders are judged on ranking, hence AUC/MRR/nDCG.

Findings

- Trained models clearly beat simple rules: best single model 63.8 vs 60.9 for the best rule (category match) and 50 for random.
- The **hybrid** (LightGBM + neural score) is best at **AUC 66.60**, 1.5 points below the paper's model, which reads full article bodies.
- Topic match alone (51.8) barely beats random: topics help as one learned signal, not as the whole ranker.
- **Limits**: offline evaluation on 2019 logs; titles and abstracts only; whether explanations improve trust needs a user study.